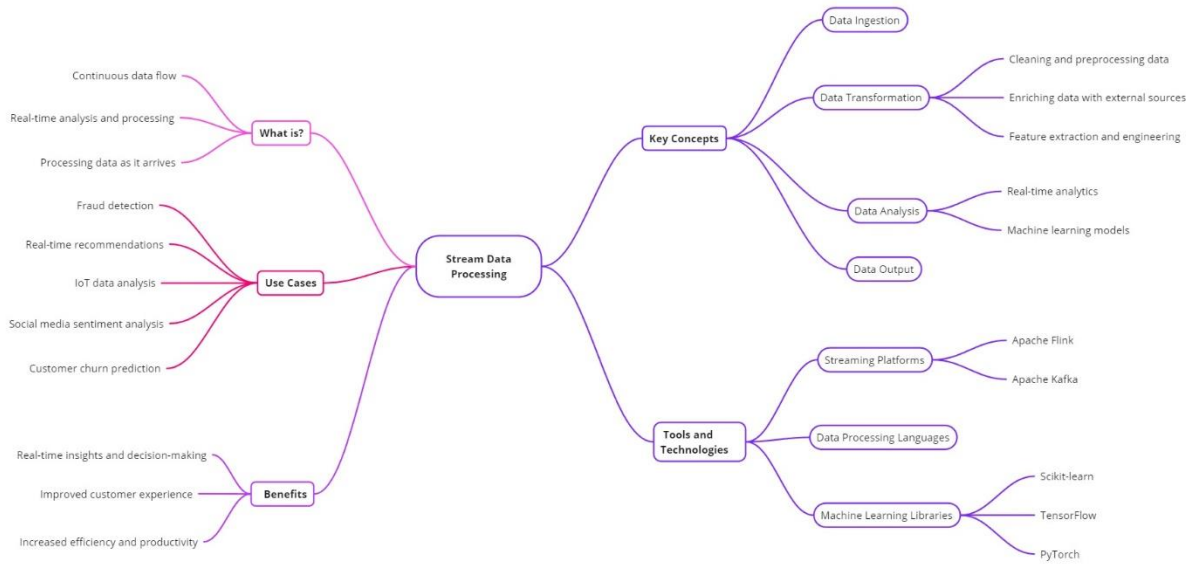
Experiment Report

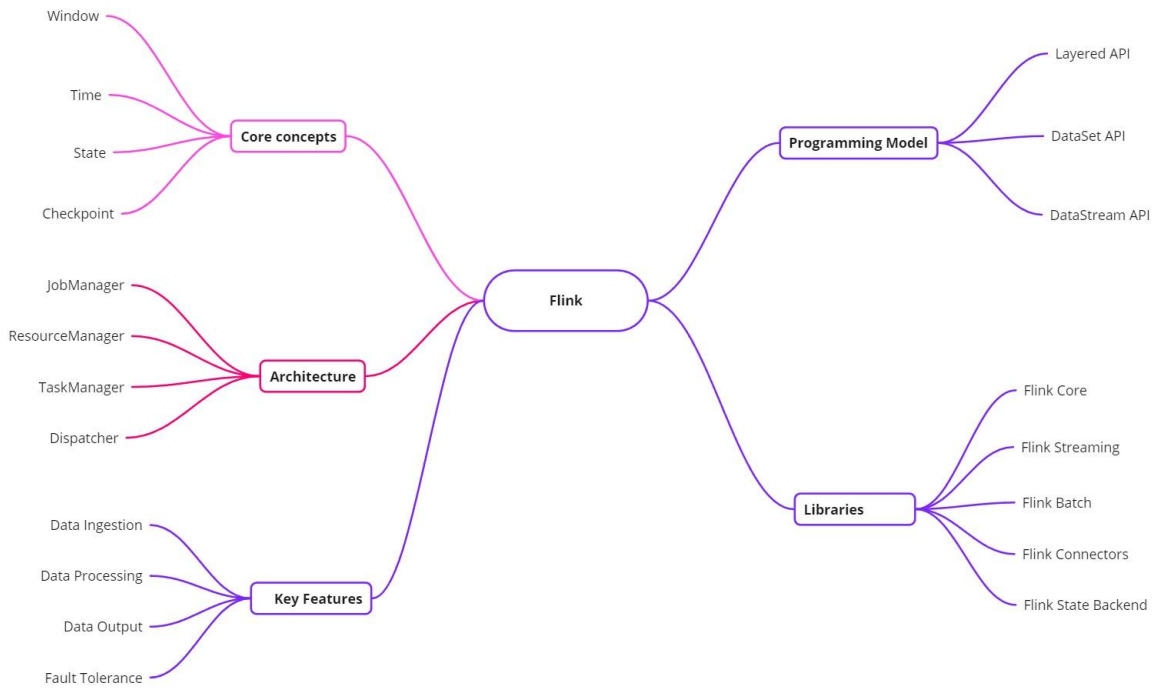
Student: Denisenko Sofiia 狄苏菲
Student id: 1820249051
Course: Big Data Analysis Technology

Experiment Topic: Stream Processing and Flink

Mind Map Stream Data Processing



Mind Map Flink



Question 1

Explain and illustrate the concepts of State and Checkpoint, as well as their working principles, and how the fault recovery can be achieved through State and Checkpoint.（Using text and the diagram）

Answer:

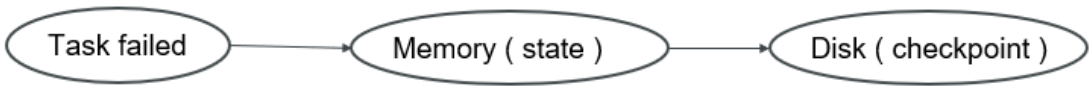
1. State:

Generally, refers to the state of a specific Task/Operator (the state of an operator indicates some intermediate results generated by some operators during operation). State data is stored in the Java heap memory/TaskManager node memory by default. State can be recorded, and data can be restored in the event of failure.

2. Checkpoint:

Represents a global state snapshot of a Flink job at a specific moment, it contains the state of all Tasks/Operators. It can be understood that Checkpoint is the timed and persistent storage of State data. Basically, Checkpoint is a snapshot of the entire Flink job's state, allowing it to recover if anything goes wrong.

Fault Recovery:



So, a Flink task encounters a problem and crashes. Before the failure, the task was diligently processing data and accumulating its state, storing it in memory. Flink already taken regular checkpoints of the job's state. The checkpoints are written to durable storage (like HDFS) to ensure they're safe even if a TaskManager crashes.

Question 2

With reference to the Flink architecture diagram, explain the components and working principles of two architectures in Flink's **YARN mode**: the task submission architecture and the runtime architecture. (Flink runtime architectures include Local mode, Standalone cluster mode, and **YARN mode**).

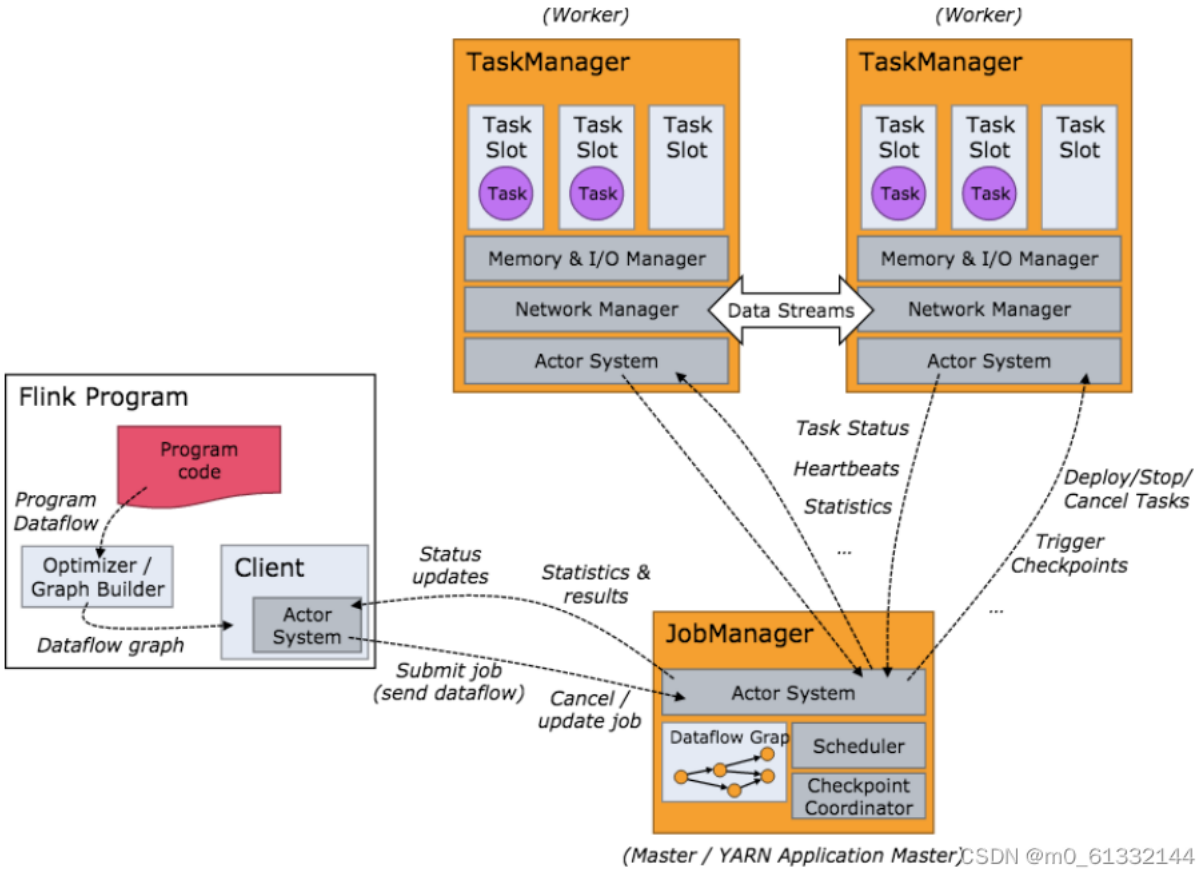
Answer:

YARN (Yet Another Resource Negotiator): YARN is a resource management system in Hadoop. It provides a framework for managing cluster resources and running applications. Flink can leverage YARN for launching and managing its applications in a distributed environment.

YARN Mode components:

- **Client**: The client first submits an application request to the YARN ResourceManager.
- **YARN ResourceManager**: YARN is responsible for managing resources and starting the Application Master (which acts as Flink's JobManager here).
- **Application Master**: Serves as Flink's JobManager in YARN, responsible for task scheduling and management.
- **TaskManager**: NodeManager in YARN starts the TaskManagers, which execute tasks and return the results.

In this mode, Flink acquires resources dynamically through YARN, making it suitable for large-scale cluster deployments.



Task Submission Architecture:

Focus:

This architecture handles the initial submission of the Flink job to the YARN cluster and the allocation of resources for the job.

Process:

1. Submission: The Flink Client sends the packaged Flink job to the YARN ResourceManager.
2. Resource Request: The client requests containers (resources) from the ResourceManager for the job.

3. Allocation: The ResourceManager allocates containers to specific NodeManagers based on availability.
4. Launch: The NodeManagers launch the Flink job (within its containers) on their respective nodes.

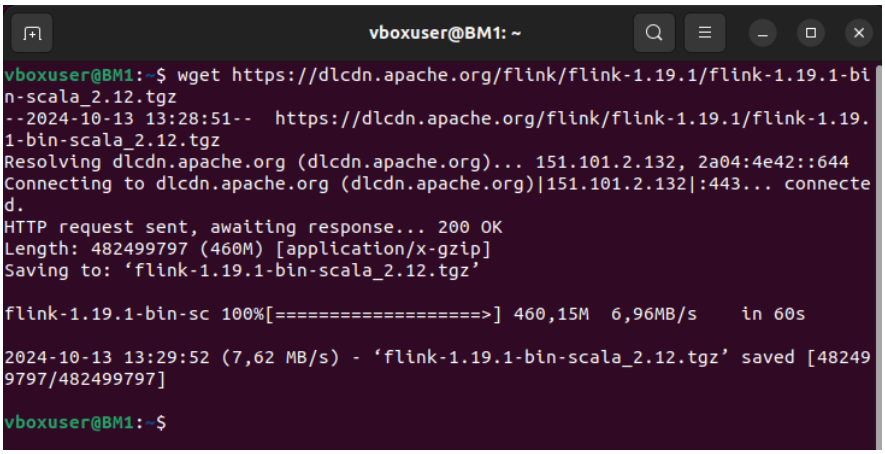
Runtime Architecture:

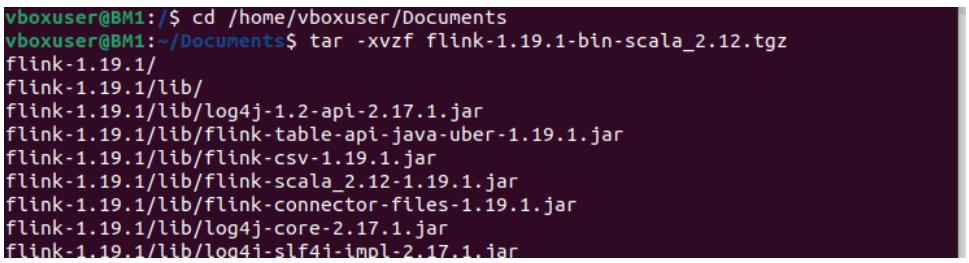
Focus:
This architecture handles the execution and management of the Flink job once it has been launched on the YARN cluster.
Process:

1. Job Initialization: The JobManager (running within a container on a YARN NodeManager) initializes the Flink job.
2. Task Deployment: The JobManager assigns tasks to different TaskManagers (also running in containers on YARN NodeManagers).
3. Data Flow: Data is processed as it flows through the tasks of the Flink job.
4. State Management: TaskManagers manage their state, periodically saving it to checkpoints for fault tolerance.
5. Fault Recovery: If a TaskManager fails, the JobManager uses the latest checkpoints to restore the state and restart the task on a different TaskManager.

Experiment 1

I had a virtual Ubuntu already installed on my pc. So, I started from the step 1.2 which is downloading apache flink.



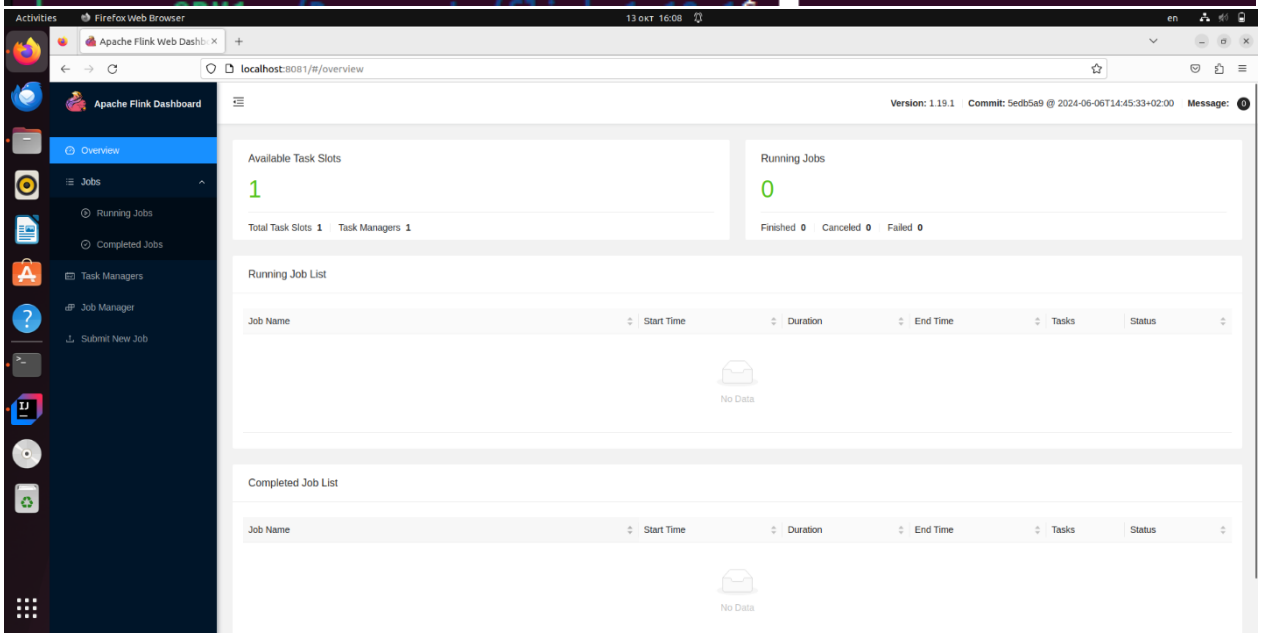


Configuring Java

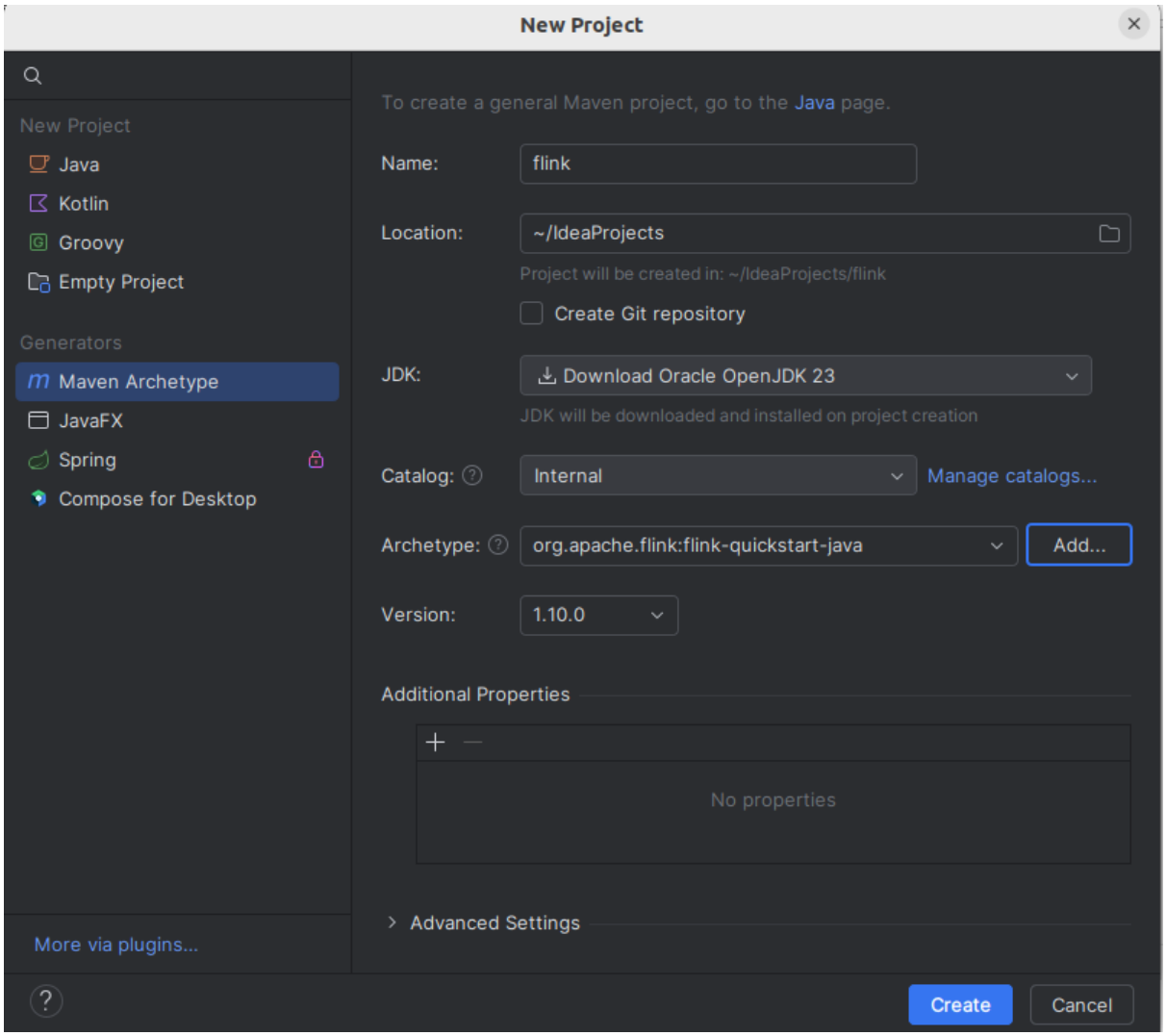
```
vboxuser@BM1:~/Documents/flink-1.19.1$ sudo apt install openjdk-17-jre-headless
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ca-certificates-java java-common
Suggested packages:
  default-jre fonts-dejavu-extra fonts-ipafont-gothic fonts-ipafont-mincho
  fonts-wqy-microhei | fonts-wqy-zenhei
The following NEW packages will be installed
  ca-certificates-java java-common openjdk-17-jre-headless
0 to upgrade, 3 to newly install, 0 to remove and 77 not to upgrade.
Need to get 48,2 MB of archives.
```

Trying to start the claster, success; then stopping claster for now.

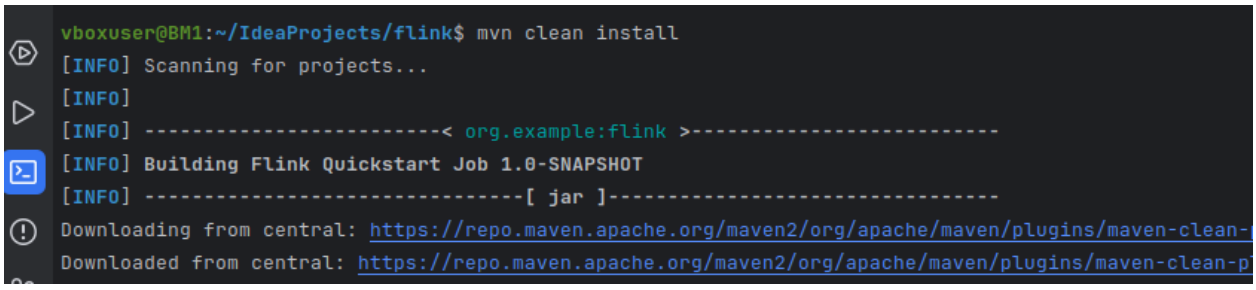
```
vboxuser@BM1:~/Documents/flink-1.19.1$ ./bin/start-cluster.sh
Starting cluster.
Starting standalonesession daemon on host BM1.
Starting taskexecutor daemon on host BM1.
```



Creating a new project in IntelliJ IDEA with Maven Archetype and adding a new archetype for Apache Flink.

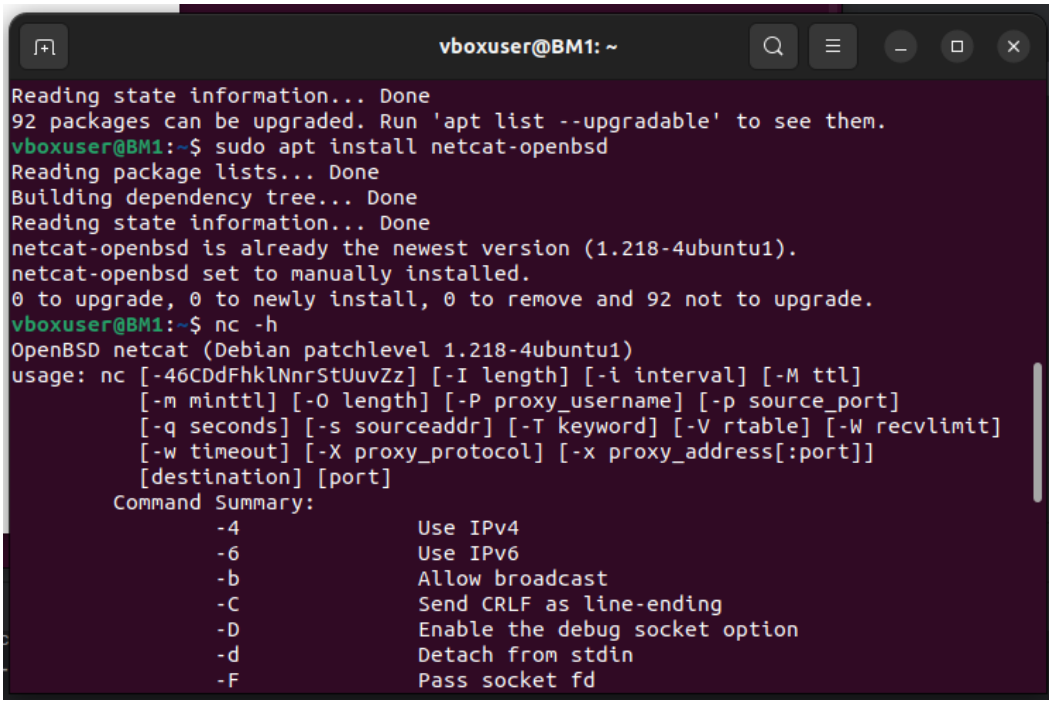


Doing the mvn clean install operation

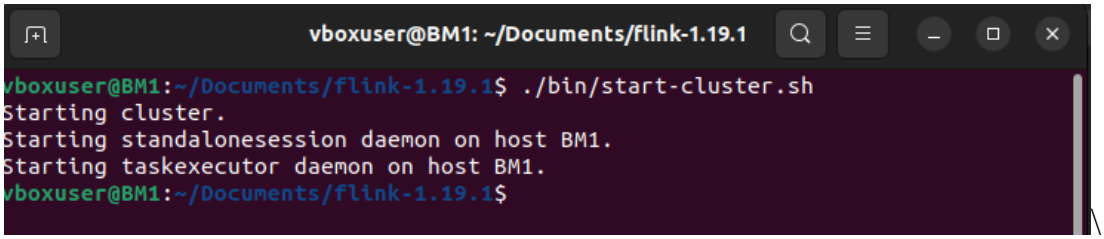


Trying to run and compile code BatchJob. Result is showed below.

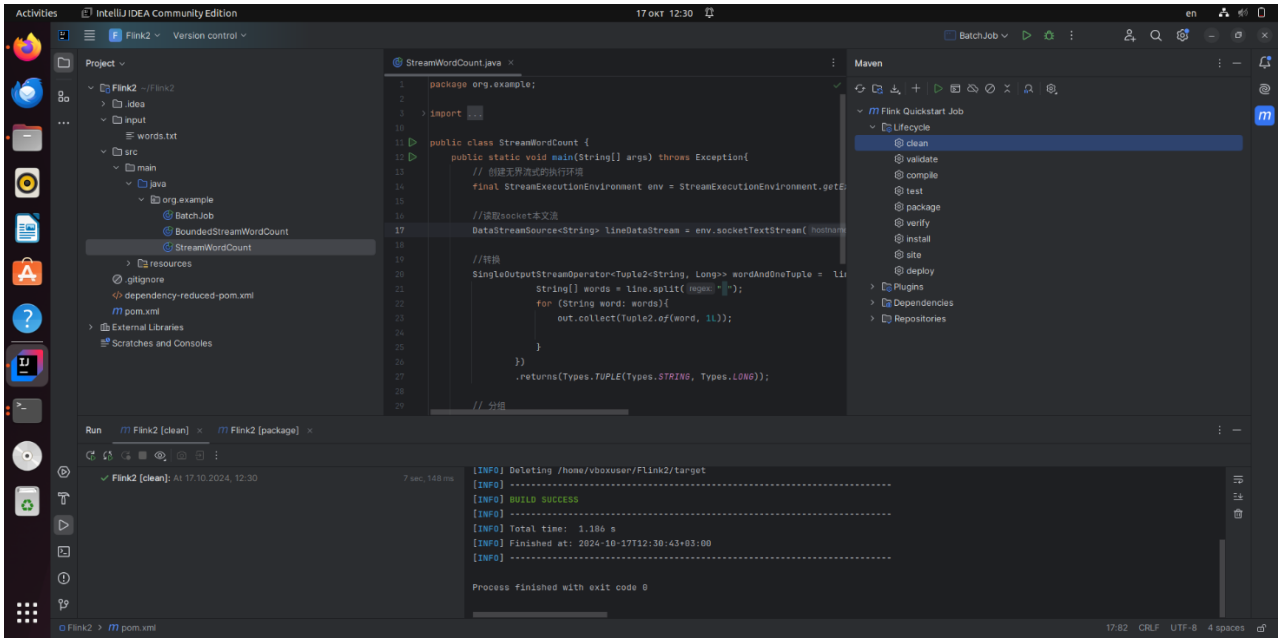
Installing netcat-openbsd

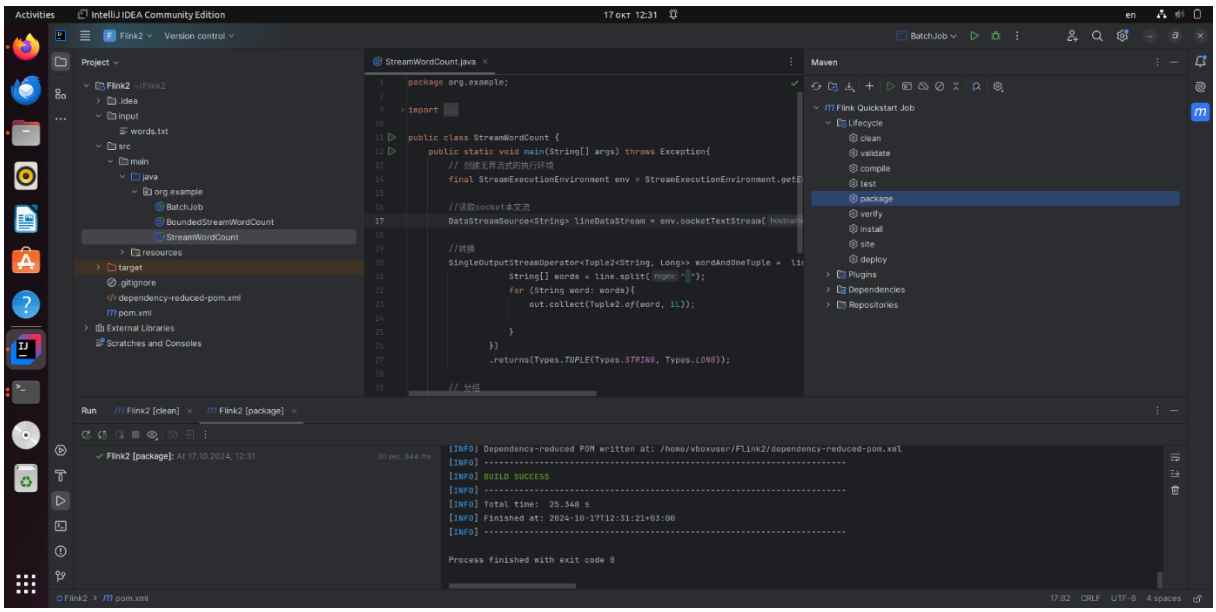


Meanwhile starting cluster.

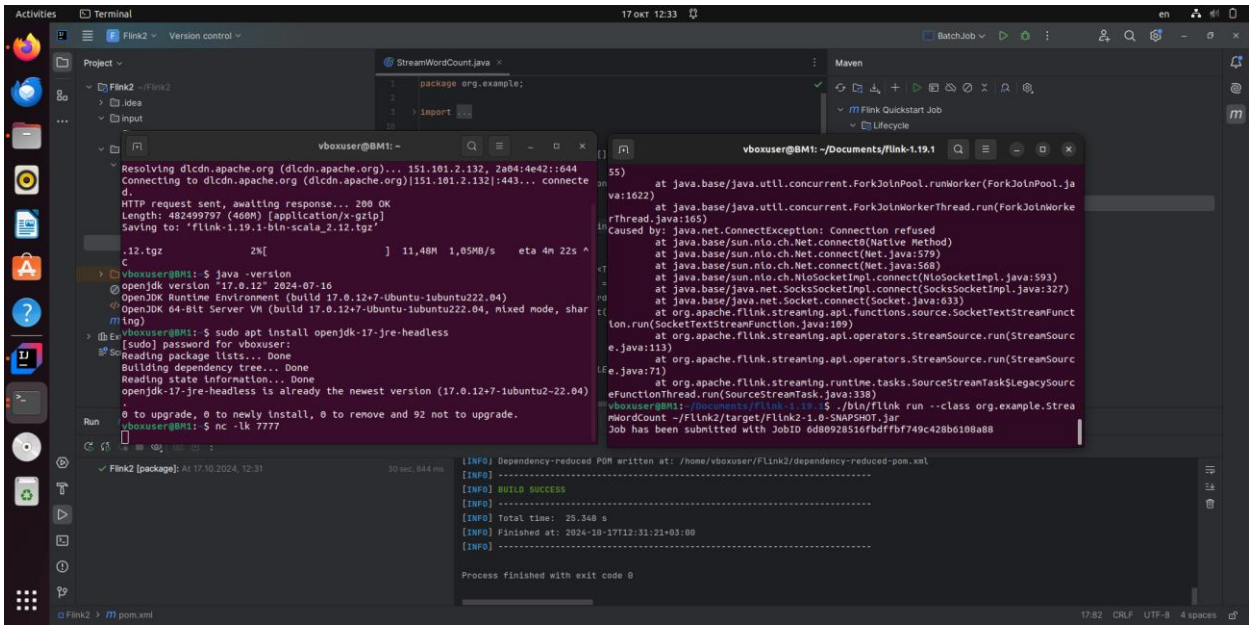


Doing clean and package for the StreamWordCount.

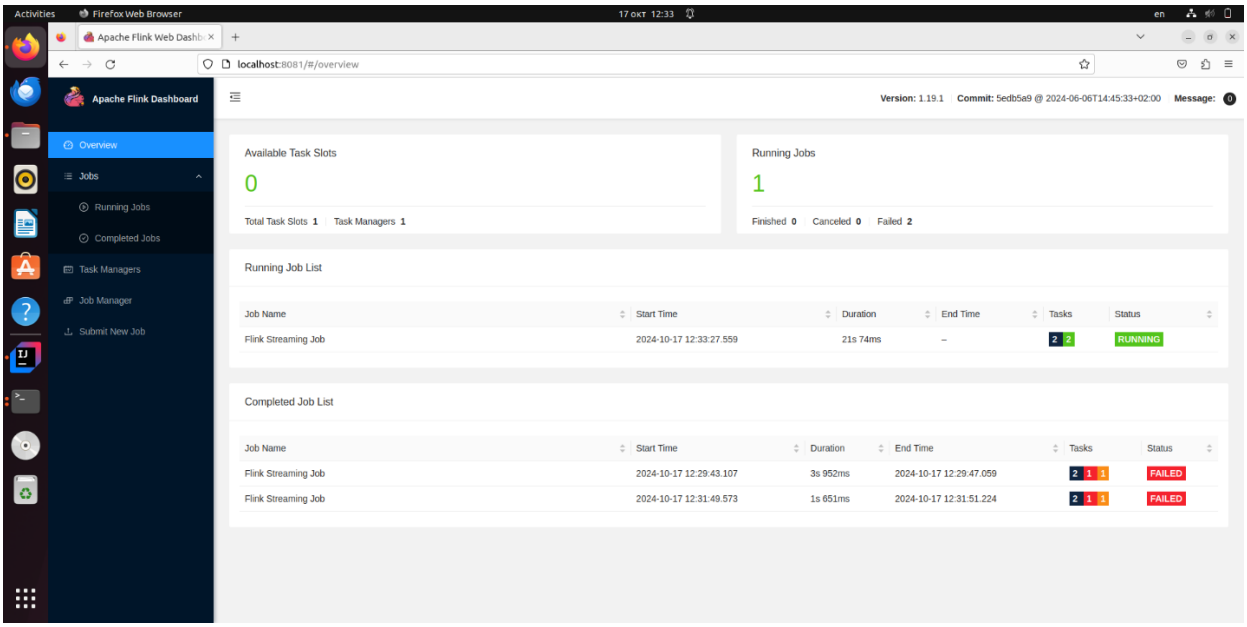


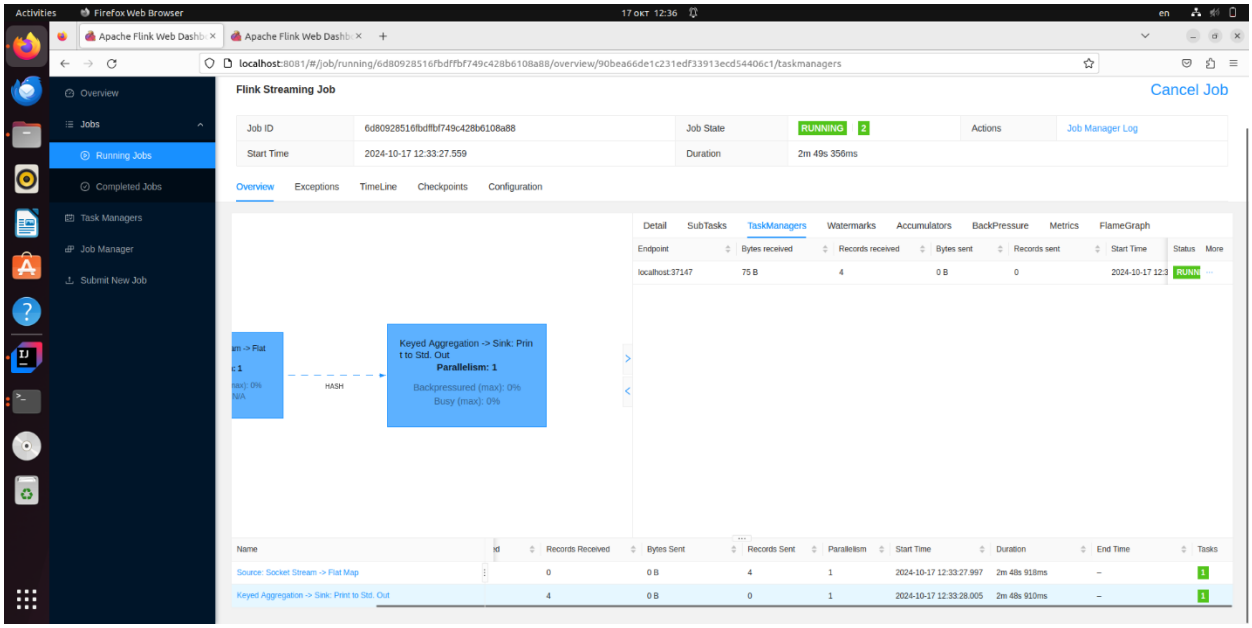


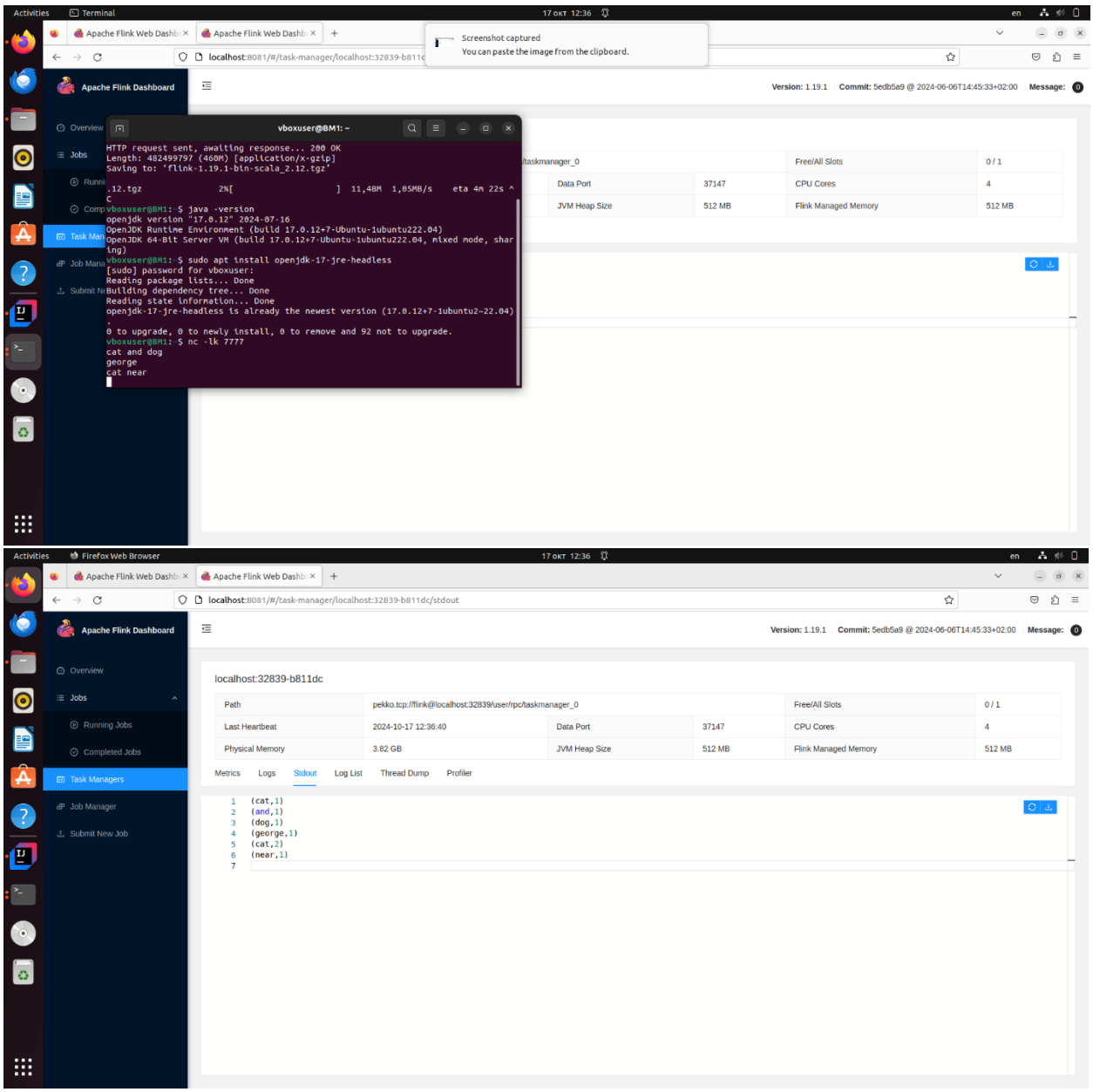
Doing the nc command and running the jar package in other terminal.



Opening localhost:8081. Opening the Flink Streaming Job. Now in stdout we can see the words I put in the terminal.







Here you can see that I put in the terminal:

cat and dog
george
cat near

And in stdout we can see:

(cat,1)
(and,1)
(dog,1)
(george,1)
(cat,2)
(near,1)